Hackathon design layout

ShopSmart AI

Autonomous e-commerce assistant platform: architecture, workflows and four independent team slices

Spring Boot 3 + Spring AI | React 19 + Vite | PostgreSQL + pgvector | RAG | Agentic AI

Lovish | Nimisha | Aman | Rajat

SCOPE

Three AI paths on one platform

1

AI product discovery

Catalog, search, filters, compare, wishlist, cart. An assistant that turns “laptop under $1000 for Java and AI” into real, explained product picks.

2

RAG policy assistant

Upload PDF, Markdown or JSON policies. Answers come only from approved documents, with source, section, version and confidence.

3

Agentic operations

Order tracking plus agents for inventory shortages, stuck orders and support triage. Every step is logged and risky actions need approval.

Grounded or silent

No answer unless an approved document supports it.

SQL-first recommendations

The LLM explains real products; it cannot invent one.

Everything auditable

Each AI call stores input, context, output and model.

STACK

Tech stack

Frontend

React 19 + Vite, React Router, TanStack Query, Tailwind CSS, Axios

Backend

Java 21, Spring Boot 3 (Web, Security + JWT, JdbcClient, Validation), springdoc OpenAPI, Maven multi-module

Database

PostgreSQL 16 with pgvector (HNSW, cosine), tsvector full-text, pg_trgm. Flyway SQL migrations are the source of truth

AI layer

Spring AI: ChatClient (any model), local ONNX embeddings all-MiniLM-L6-v2 (384-dim), pgvector SQL (cosine), typed JSON output with bean validation

Async and infra

@Async ingestion and @Scheduled mock delivery events (no Redis or broker); Docker Compose for one-command setup

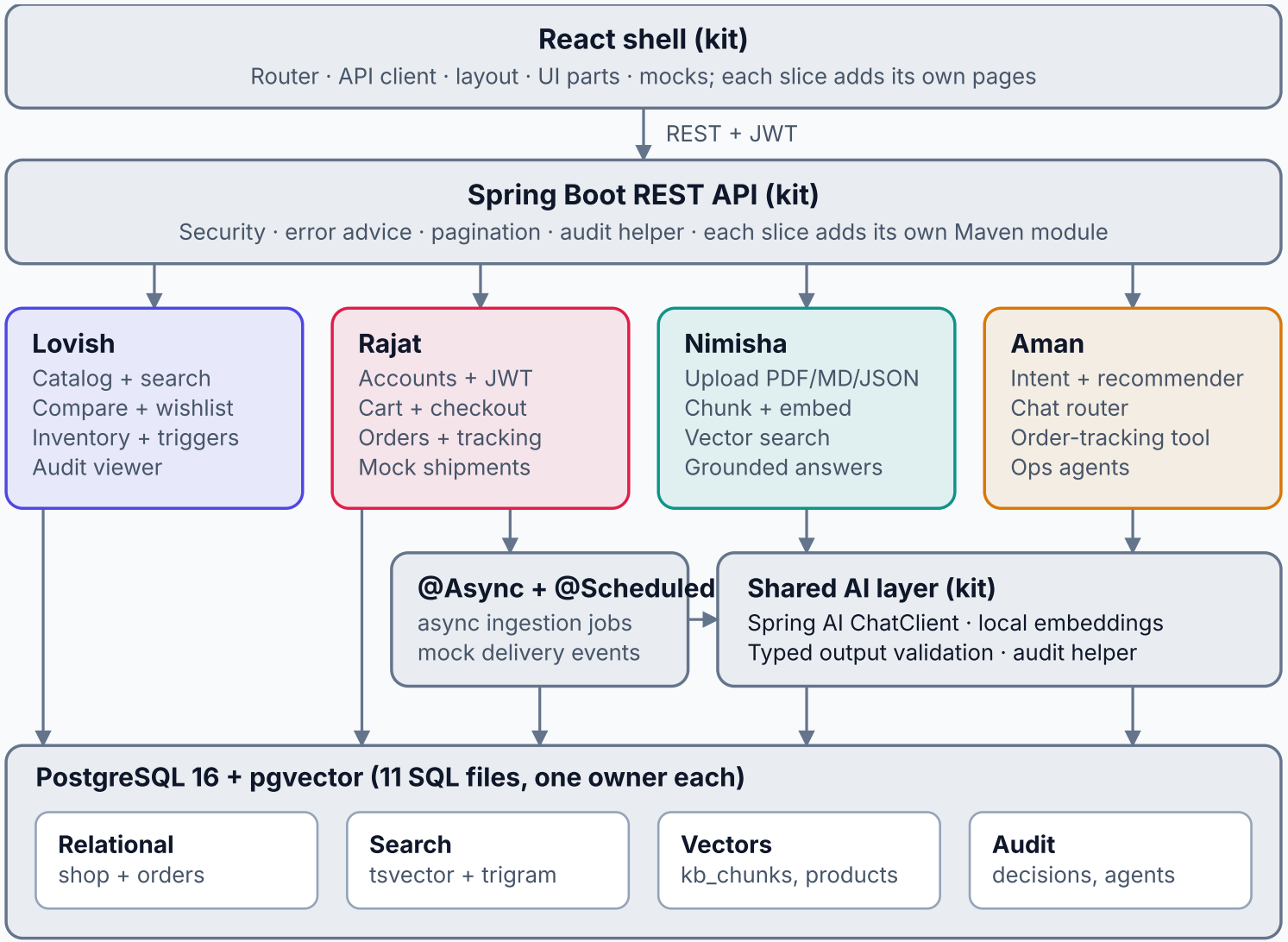
Quality

Bean Validation, one @ControllerAdvice error format, JUnit 5 + Testcontainers, Vitest, 15-question RAG test set

This is the stack recommended in the problem statement. Any chat model plugs in through Spring AI.

ARCHITECTURE

System architecture



HOW TO READ IT

- The browser only talks to the Spring Boot API.
- Four coloured slices sit on a grey shared kit.
- Async jobs run slow work, so uploads return instantly.
- One PostgreSQL holds relational data, search and vectors.

BOX COLOUR = OWNER

- **Lovish** Catalog, inventory, audit viewer
- **Rajat** Accounts, cart, orders
- **Nimisha** RAG service
- **Aman** Assistant, agents
- **Kit** shared, built in Phase 0, then frozen

DATABASE

Data model: 28 tables in 11 SQL files

Lovish

`10_catalog.sql` categories, brands, products, product_images
`11_inventory.sql` warehouses, inventory, inventory_movements, 2 views
`21_wishlist.sql` wishlist_items
`23_inventory_triggers.sql` take stock on order item, restock on cancel
`40_audit.sql` ai_decision_logs

Rajat

`20_accounts.sql` users, addresses, preferences (3 tables)
`22_cart_orders.sql` carts, cart_items, orders, order_items, shipments, shipment_events, v_order_tracking

Nimisha

`30_knowledge.sql` kb_documents, kb_chunks (vector + full-text)

Aman

`50_assistant.sql` chat_sessions, chat_messages
`60_agents_ops.sql` agent_runs, agent_steps, stock_alerts, support_tickets

One writer per table

Others read tables or views. The stock trigger, not orders, moves inventory.

SQL is the source of truth

Flyway runs the SQL; JdbcClient queries it directly; tests use pgvector Testcontainers.

Views are the read contract

v_product_availability, v_sales_velocity, v_order_tracking.

Workflow 1: AI shopping assistant

1. **Ask**
   User: “Laptop under $1000 for Java and AI projects”
2. **Extract intent**
   LLM returns JSON: category, budget, use case, specs
3. **Clarify?**
   If budget or category is missing, return a clarifying question
4. **Search in SQL**
   Filter category, price, stock; rank by text, specs, rating, preferences
5. **Explain**
   LLM sees only the top 3 to 5 real rows and writes a reason each
6. **Respond + log**
   Brief’s JSON returned; row saved in ai_decision_logs

```
{
  "interpretedRequirement": "Laptop for Java development and AI learning",
  "budget": 1000,
  "recommendations": [{ "productId": "LAP101", "productName": "Dell Inspiron 15",
     "reason": "16GB RAM, Intel i7 processor, SSD storage", "availability": "IN_STOCK" }],
  "clarifyingQuestion": null
}
```

Workflow 2: RAG knowledge assistant

INGESTION (ADMIN, ASYNC)

1. **Upload**
   PDF, MD or JSON + version, date, category, region
2. **Parse + chunk**
   Split by headings, ~500 tokens, keep section title
3. **Embed**
   Local 384-dim embeddings, stored in kb_chunks
4. **Approve**
   Status APPROVED; older version SUPERSEDED

QUESTION ANSWERING

1. **Embed question**
   e.g. “Can I return an opened laptop?”
2. **Filtered search**
   Approved docs only, user region, effective date, top 5
3. **Threshold**
   Weak match: say “not in approved documents”
4. **Grounded answer**
   LLM answers from context and cites chunks
5. **Return + log**
   answer, confidence, sources[], followUpRequired

Workflow 3 and 4: order tracking and ops agents

ORDER AND SHIPMENT TRACKING

1. Chat router detects an order question (“Where is order 1042?”).
2. Tool reads v_order_tracking, only for the logged-in user’s orders.
3. Reply: status, last event, ETA, delay reason.
4. Policy parts (“Can I cancel after shipment?”) go to PolicyAnswerService.
5. Rajat’s @Scheduled job advances mock shipments and writes events, so the demo shows movement.

OPS AGENTS (BOUNDED TOOL LOOP)

Inventory agent
Reads v_sales_velocity, raises stock alerts, proposes restock or transfer.

Fulfilment agent
Finds orders stuck too long, checks stock, reroutes or opens a logistics ticket.

Support triage agent
Classifies a complaint, answers policy questions via RAG, else opens a ticket.

Allow-listed tools, step limit, every step stored in agent_steps. Agents propose; a human approves.

COLLABORATION

How the four slices stay independent

1. **One owner per folder and SQL file**
   A CODEOWNERS file makes pull requests enforce it.

2. **One writer per table**
   Others only read, directly or through a view.

3. **Talk only through frozen contracts**
   Tables, views, each person's OpenAPI file, and one Java interface, PolicyAnswerService.

4. **Every slice ships its own sample data**
   db/seed/<NN>_<slice>.sql, so nobody waits for another slice.

5. **A shared kit, then frozen**
   Parent POM, error advice, security/JWT, AI gateway, audit helper, React shell. Controllers and routes are auto-discovered, so nobody edits a shared file.

Each person owns one vertical slice (Flyway files, Maven module, React folder, sample data, tests), so a slice runs and tests on its own.

COLLABORATION

Repo layout and ownership

`shopsmart-ai/`	
`pom.xml, docker-compose.yml, .env.example, backend/kit/, app/`	Lovish
`CODEOWNERS, PolicyAnswerService stub, CI`	Aman
`kit/ai (LlmGateway, embeddings, AuditLogger)`	Nimisha
`frontend/src/shell, api, ui, mocks`	Rajat
`backend/catalog + features/catalog`	Lovish
`backend/commerce + features/account`	Rajat
`backend/knowledge + features/knowledge`	Nimisha
`backend/assistant + features/assistant`	Aman

WORKING RULES

- Kit pieces are built in Phase 0 (about 3 hours each) and frozen after.
- Spring finds each module's controllers by component scan and React finds `features/*/routes.jsx` automatically.
- Everyone writes `docs/<name>.md` and `docs/contracts/<name>.yaml` for their own slice.
- Small branches; one other person reviews each pull request.
- A kit change needs a small pull request that the other three approve.

COLLABORATION

Contracts between slices

From	To	Contract	Until the real one lands
Rajat cart + checkout	Lovish stock	Reads v_product_availability; stock trigger moves stock	Own 5-product sample data
Lovish product page	Rajat cart	POST /api/cart/items/	Mock handler
Aman recommender	Lovish, Rajat	Reads products, availability view, customer_preferences	Own sample data
Aman order tool, fulfilment agent	Rajat orders	Reads v_order_tracking	Own sample data
Aman inventory agent	Lovish stock	Reads v_sales_velocity; writes only stock_alerts	Own sample data
Aman chat router	Nimisha RAG	PolicyAnswerService.ask(question, user) returns answer, confidence, sources, followUpRequired	@ConditionalOnMissingBean stub in kit
Nimisha, Aman	Lovish audit viewer	audit.log_decision() writes ai_decision_logs	Helper in kit from day 1
Lovish product page	Aman chat widget	Browser event assistant:open; cards link to /products/:sku	Ignored if absent

Each side builds against the contract and its own stand-in, so integration is a one-line switch in Phase 3.

TEAM

Who builds what

Lovish

Catalog, inventory and audit viewer

~29 h + 3 h kit

modules: catalog, inventory, audit

SQL: 10, 11, 21, 23, 40

Kit: parent POM, Docker, Flyway setup, dev login

Rajat

Accounts, cart and orders

~30 h + 3 h kit

modules: accounts, cart, orders

SQL: 20, 22

Kit: React shell, API client, UI parts, mocks

Nimisha

RAG knowledge assistant

~31 h + 3 h kit

modules: knowledge

SQL: 30

Kit: Spring AI config, embeddings, audit helper

Aman

AI assistant and ops agents

~31 h + 3 h kit

modules: assistant, agents, ops

SQL: 50, 60

Kit: PolicyAnswerService stub, CODEOWNERS, CI

Every slice mixes backend, React and data work, and hours are within about 7% of each other.

TEAM MEMBER TASKS

Lovish · Catalog, inventory and audit viewer

L

29 hslice + 3 h kit

OWNS

Spring modules: catalog, inventory, audit
React: catalog, inventory, audit
SQL: 10, 11, 21, 23, 40

READS ONLY

users (audit filter), order_items (trigger fires on insert)

DONE WHEN

Browse, compare and wishlist work on his sample data; an oversell is rejected; the audit viewer lists decisions.

- [ ] Catalog entities, Flyway files, OpenAPI file, sample data: 6 categories, 100+ products with specs and images **4 h**
- [ ] Products API: full-text and typo-tolerant search, filters, sorting, pagination, detail **4 h**
- [ ] Compare endpoint and wishlist API **2 h**
- [ ] Inventory entities, staff inventory API (per warehouse, low stock), tests for the stock triggers **3 h**
- [ ] Audit viewer API: list and detail, filters by task type, user, date **2 h**
- [ ] React home + catalog: category tree, search, filters, sort, pagination, responsive grid **4 h**
- [ ] React product page (gallery, specs, availability, add to cart, wishlist, Ask AI) + compare view **4 h**
- [ ] React wishlist, staff inventory page, audit viewer showing the retrieved context **4 h**
- [ ] Slice tests and docs/lovish.md **2 h**

TEAM MEMBER TASKS

Rajat · Accounts, cart and orders

R

30 hslice + 3 h kit

OWNS

Spring modules: accounts, cart, orders
React: account, cart, orders
SQL: 20, 22

READS ONLY

products, v_product_availability

DONE WHEN

A new user can register, add to cart, check out and watch the order move through mock shipping.

- [ ] User entity, register / login / refresh / me with Spring Security JWT, tiers, OpenAPI file **3 h**
- [ ] Preferences and addresses APIs **2 h**
- [ ] Cart API with validation (quantity, stock from v_product_availability) **3 h**
- [ ] Checkout in one transaction: order, items, shipment, snapshots; INSUFFICIENT_STOCK becomes a clear 409 **4 h**
- [ ] Order list, detail, cancel (blocked after shipment), tracking endpoint on v_order_tracking **3 h**
- [ ] Mock shipment engine: @Scheduled job advances shipments, writes events, simulates a delay **3 h**
- [ ] Sample data: a demo user per tier, staff user, addresses, orders in every status **1 h**
- [ ] React login, register, profile with preferences and addresses **4 h**
- [ ] React cart and checkout **4 h**
- [ ] React orders list + detail with tracking timeline, tests, docs/rajat.md **3 h**

TEAM MEMBER TASKS

Nimisha · RAG knowledge assistant

N

31 hslice + 3 h kit

OWNS
Spring module: knowledge
React: knowledge
SQL: 30

READS ONLY
products (manual links), users (uploaded_by)

DONE WHEN
The five example questions return cited answers; out-of-scope questions are refused; the 15-question set passes.

- [ ] Entities, OpenAPI file, multipart upload API with validation; checksum rejects duplicates **3 h**
- [ ] Parsers for PDF (PDFBox/Tika), Markdown and JSON, heading-aware chunker with overlap **4 h**
- [ ] Ingestion: embeddings into pgvector, @Async job, status updates, failure handling **3 h**
- [ ] Retrieval: cosine search over approved chunks, region and date filters, threshold **4 h**
- [ ] Grounded answer: prompt, citations to sources[], confidence, followUpRequired, refusal **4 h**
- [ ] Versioning, document list / patch / delete, real PolicyAnswerService bean **3 h**
- [ ] Sample knowledge base: 8 to 10 policy docs in PDF, MD, JSON + 15-question eval set **3 h**
- [ ] React admin: upload with metadata form and progress, list with status and versions **3 h**
- [ ] React policy Q&A page: answer card, confidence badge, source chips, refusal state **3 h**
- [ ] Slice tests and docs/nimisha.md **1 h**

TEAM MEMBER TASKS

Aman · AI assistant and ops agents

A

31 hslice + 3 h kit

OWNS

Spring modules: assistant, agents, ops
React: assistant, agents
SQL: 50, 60

READS ONLY

products, v_product_availability, customer_preferences, v_order_tracking, v_sales_velocity, PolicyAnswerService

DONE WHEN

Recommendation JSON matches the brief; chat routes four intents; agents raise alerts and tickets with every step logged.

- [ ] OpenAPI file, intent extraction prompt, typed record output, validation and retry **3 h**
- [ ] SQL-first recommender: filter by category, price, stock; rank by text, specs, rating, preferences **4 h**
- [ ] POST assistant/recommend/ with the brief's exact JSON, reasons, clarifying question, audit row **3 h**
- [ ] Chat router: shopping, policy (PolicyAnswerService), order (v_order_tracking), support; session storage **3 h**
- [ ] Agent runner: allow-listed tools, step limit, agent_steps, approval gate **3 h**
- [ ] Inventory and fulfilment agents using v_sales_velocity and v_order_tracking; write stock_alerts **4 h**
- [ ] Support-triage agent and tickets API **3 h**
- [ ] React chat widget: product cards with reasons, clarifying prompts, sources, order status card **4 h**
- [ ] React admin: agent runs with steps and approve, tickets, stock alerts **3 h**
- [ ] Slice tests and docs/aman.md **1 h**

PLAN

Build order and gates

0 Shared kit

All four, ~3 h each: parent POM + Docker + Flyway + dev login; React shell + mocks; Spring AI config + audit helper; PolicyAnswerService stub + CODEOWNERS + CI. Each commits an OpenAPI file.

Gate: docker compose up runs all migrations; kit frozen

1 Slice backend

Each alone: own models, APIs, sample data and tests, using only the kit.

Gate: Slice tests pass with no other slice running

2 Slice frontend

Each alone: own React features against the mock handlers.

Gate: Every feature works on mocks

3 Integration

All four, 1 to 2 h: mocks off, real PolicyAnswerService on, all sample data loaded, demo script run. Bugs go to the folder owner.

Gate: Demo runs end to end

4 Polish

Validation, error states, 15-question RAG check, mobile pass, README, rehearsal.

Gate: Demo runs twice with no edits

Only Phase 0 and Phase 3 need all four of you together. Split your hackathon hours across the phases, for example 10% / 55% / 15% / 20%.

FINISH LINE

Demo script and submission checklist

DEMO (ABOUT 5 MINUTES)

1. Log in as a Prime user; filter, sort and compare two laptops.
2. Ask for a laptop under $1000; then a vague phone request to show the clarifying question.
3. “Can I return an opened laptop?” with sources and confidence; then an out-of-scope question.
4. Upload a new policy version and ask again.
5. Place an order and ask “Where is my order?”
6. Run the inventory agent; show its steps in the audit screen and approve the proposal.

SUBMISSION CHECKLIST

- [ ] Working app and source code; one docker compose up
- [ ] Architecture diagram and solution overview
- [ ] Setup instructions, .env keys, test users per tier
- [ ] Live demonstration of the six steps
- [ ] Flyway migrations apply cleanly on pgvector/pg16
- [ ] Validation and error handling, including model or DB down
- [ ] Clean folder structure
- [ ] Audit screen explains any AI answer